

CINEMAX – QUADRO DI INTEGRAZIONE TECNICA

SPECIFICHE ARCHITETTURALI E STANDARD DI SVILUPPO DEL SOFTWARE

Autore: Morad (Project Manager)
Destinatari: Team di Sviluppo (Giovanni, Harman, Ayman)
Versione: 1.0 (Definitiva)
Timeframe Rilascio: 1 Settimana (Rilascio Singolo)

Il presente documento tecnico stabilisce in modo stringente l'architettura dei moduli, l'organizzazione dei package strutturali, le interfacce di comunicazione interne e gli standard di robustezza richiesti per il corretto completamento del progetto CineMax. Avendo a disposizione una singola settimana per la finalizzazione e l'unione dei componenti, l'aderenza a queste specifiche è da considerarsi tassativa per evitare conflitti e garantire il superamento dell'esame.

1. Struttura dei Package e Directory del Repository

Il codice sorgente dell'applicazione Java deve essere organizzato seguendo rigorosamente l'albero strutturale descritto di seguito. È fatto assoluto divieto di inserire o istanziare classi al di fuori dei perimetri designati:

```
cinemax/
|
|— CineMax.java           # Entry point dell'applicazione (contiene il metodo main)
|
|— tui/                   # LAYER PRESENTATION: Gestione dell'interfaccia a
|  terminale
|  |— MenuManager.java    # Gestione dei loop dei menu operativi e navigazione ad
|  |  albero
|  |  |— InputReader.java  # Utility centralizzata per acquisizione e convalida input
|  |  |  da tastiera
|  |  |— ViewPrinter.java  # Formattazione grafica standard (tabelle, messaggi,
|  |  layout CLI)
|
|— core/                  # LAYER BUSINESS LOGIC: Modelli e servizi applicativi
|  |— auth/
|  |  |— AuthenticationService.java # Gestione login, controllo ruoli e sessione attiva
|  |  |— services/
|  |  |  |— PalinsestoService.java # Manipolazione palinsesto, film e pianificazione
|  |  |  |  sale
|  |  |  |  |— BookingService.java # Elaborazione prenotazioni e calcolo disponibilità
|  |  |  |  |  posti
|  |  |  |  |  |— data/
|  |  |  |  |  |  |— csv/
|  |  |  |  |  |  |  |— CsvEngine.java # Motore generico di lettura/scrittura tokenizzata
|  |  |  |  |  |  |  |— RecordMapper.java # Serializzatore/Deserializzatore da oggetto a stringa CSV
```

2. Standard di Comunicazione e Firme Metodi (API Interne)

I componenti software implementati dai singoli sviluppatori devono integrarsi in modo trasparente attraverso contratti di interfaccia stabili. Di seguito sono specificate le firme dei metodi principali che collegano il layer di presentazione con la business logic:

Modulo Autenticazione (AuthenticationService.java)

Consente la verifica e la contestualizzazione dell'utente all'interno della sessione di runtime dell'applicazione.

```
/**
 * Esegue la verifica delle credenziali d'accesso confrontandole con i record cifrati.
 * * @param username Identificativo stringa inserito dall'utente
 * @param password Password in chiaro da sottoporre ad hashing preventivo
 * @return Oggetto Utente completo se autenticato, null in caso di credenziali errate
 */
public Utente eseguiLogin(String username, String password);
```

Modulo Prenotazioni (BookingService.java)

Elabora in modo transazionale le transazioni d'acquisto dei biglietti, applicando le verifiche bloccanti di capienza.

```
/**
 * Registra una nuova prenotazione verificando in tempo reale la capienza residua.
 * * @param utente Il cliente registrato che esegue l'operazione
 * @param proiezione La proiezione del palinsesto cinematografico selezionata
 * @param postiRichiesti Numero di poltrone da riservare in sala
 * @return String codice univoco alfanumerico della prenotazione generata
 * @throws IllegalArgumentException se i posti richiesti superano la capienza residua
 */
public String creaPrenotazione(Utente utente, Proiezione proiezione, int postiRichiesti)
throws IllegalArgumentException;
```

3. Flusso di Lavoro GitHub e Integrazione Finale

Lavorando all'interno di un arco temporale ridotto, la gestione dei rami di sviluppo deve essere finalizzata alla minimizzazione dei rischi di disallineamento e alla perdita di codice:

- **Branch Portanti Protetti:** È fatto esplicito divieto di eseguire commit diretti sui branch `main` e `develop` sul repository remoto di GitHub.
- **Sviluppo in Isolamento (Feature):** Ogni sviluppatore deve avviare il proprio task atomico staccando un branch locale derivato da `develop` (es. `feature/csv-parser`, `feature/tui-menu`).
- **Integrazione Giornaliera:** Al completamento funzionale di un componente, viene aperta una Pull Request verso il branch `develop`. Il Project Manager effettua la code review e approva il merge sul server di integrazione solo in seguito a una compilazione locale andata a buon fine.

Robustezza del Sistema: Eventuali anomalie di I/O nei file gestiti sul file system locale non devono determinare l'arresto imprevisto dell'applicazione. Tali eccezioni devono essere gestite all'interno del layer di appartenenza e propagate alla TUI sotto forma di messaggi d'errore parlanti per l'operatore, mantenendo inalterato il ciclo attivo dei loop dei menu.